SDL2 Learning Jumpaverse

Author:ERROR_343(Muhammad Awais),Ahmed Hamza,Hamza Khan

Author:ERROR_343(Muhammad Awais),Ahmed Hamza,Hamza Khan

Jumpa Verse SDL2 Game Documentation

This document explains the current C++ SDL2 game code from basic SDL concepts up to the project structure, object model, rendering pipeline, procedural ground generation, menu system, traps, camera movement, and cleanup. The examples are intentionally simple so a beginner can understand each part before reading the actual project files.

Project: Jumpa-Verse | Main language: C++ | Graphics library: SDL2 | Theme: Orange

1. Project Overview

Jumpa Verse is an SDL2 side-scrolling platform-style game prototype. SDL2 does not provide a built-in 2D camera, so this project creates the camera illusion by changing object x positions over time. Platforms and traps move left across the fixed window while the player would appear to move through the world.

- main.cpp owns the main game loop, timing, texture loading, font loading, rendering order, and shutdown.
- object.h defines the object structures: hitBox, ground_Class, trapSpike, mainMenuColor, and optionBoxes.
- ground.cpp creates the first platform, generates later random platforms, and removes old ground.
- traps.cpp creates triangular spike traps using SDL2_gfx and frees old trap objects.
- camera.cpp moves ground and trap positions to simulate camera movement.
- menuScreen.cpp renders the menu buttons and handles Play/Exit mouse input.
- Error_Handlers.cpp contains SDL, SDL_image, and SDL_ttf initialization plus cleanup.

2. File Structure

```
Jumpa-Verse/
main.cpp                -> program entry point and game loop
object.h                -> game classes and object data structures
functions.h             -> shared function declarations
Error_Handlers.cpp      -> SDL initialization and shutdown
ground.cpp              -> random ground generation and ground cleanup
traps.cpp               -> spike trap generation and trap cleanup
camera.cpp              -> camera illusion by moving object positions
menuScreen.cpp          -> Play and Exit menu
Images_textures/        -> platform images
debrosee-font/          -> TTF font used by menu text
Tutorial_Image/         -> SDL coordinate and triangle reference images
```

3. UML Object Structure

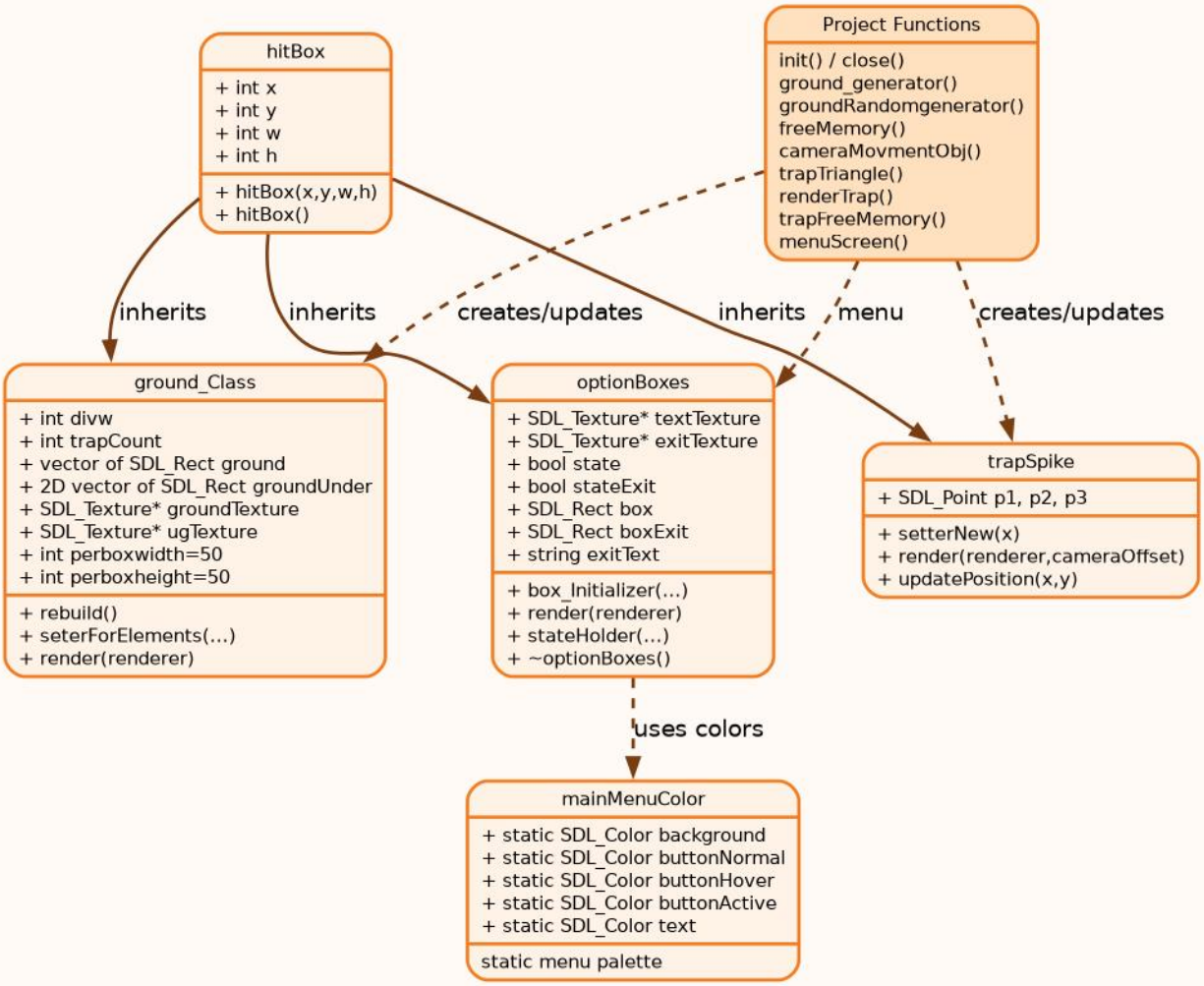

Figure 1: UML class and function relationship diagram

The UML diagram shows that hitBox is the base rectangle structure. Ground platforms, spike traps, and option boxes inherit the same x, y, w, and h fields. This is useful because every visible object needs a position and size.

3.1 Class Details

hitBox

Purpose: stores the rectangle information common to game objects.

```
class hitBox {
public:
int x, y, w, h;
hitBox(int x, int y, int w, int h);
hitBox();
};
```

ground_Class

Purpose: represents one platform. The top is stored in ground, and the underground body is stored in groundUnder. It uses shared textures, meaning the class does not load or own the image files itself.

- divw: number of 50-pixel columns across the platform width.
- trapCount: number of traps currently created on that platform.

- ground: one-dimensional vector of SDL_Rect top tiles.
- groundUnder: two-dimensional vector of SDL_Rect lower tiles.
- rebuild(): recalculates every tile rectangle when x/y/w/h changes.
- seterForElements(): changes platform bounds and rebuilds geometry.
- render(): draws the top and underground tiles with SDL_RenderCopy.

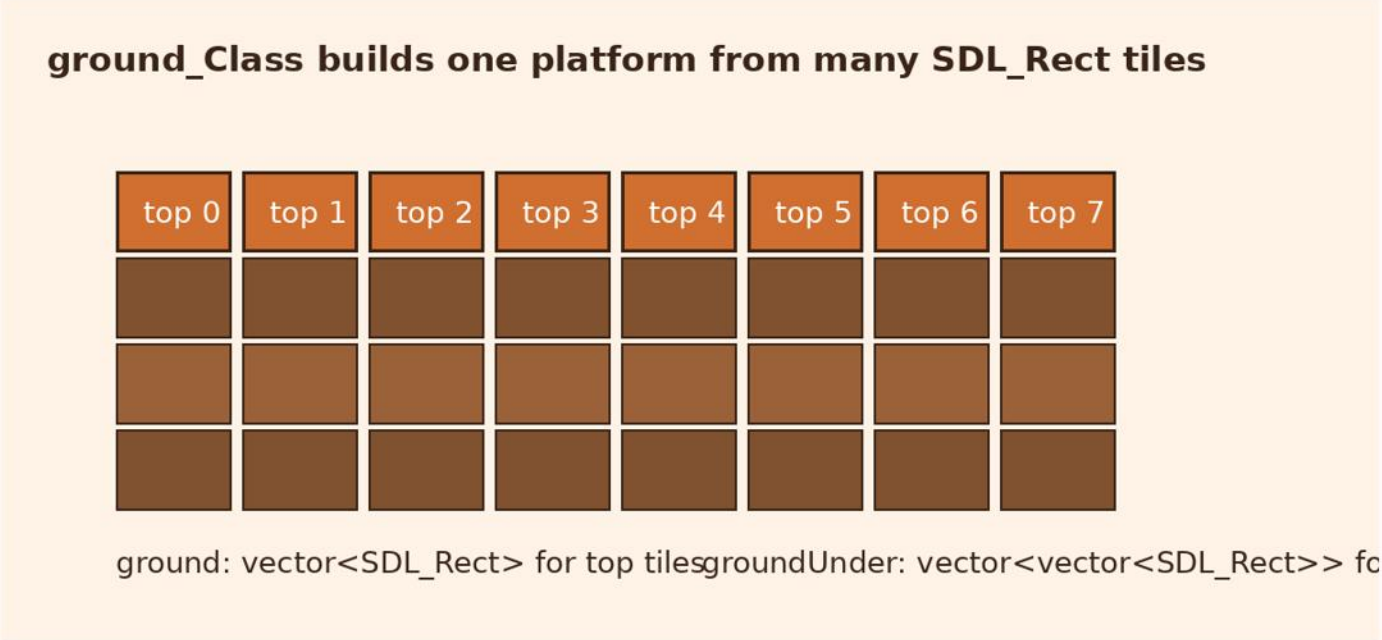

Figure 2: How ground_Class breaks one platform into SDL_Rect tiles

trapSpike

Purpose: represents one triangular spike. It inherits the hitBox rectangle but draws itself using three SDL_Point values. SDL2_gfx functions filledTrigonRGBA and trigonRGBA draw the filled triangle and outline.

```
trapSpike(int x, int y, int w, int h) : hitBox(x, y, w, h) {
p1 = {x, y};
p2 = {x + 50, y};
p3 = {x + 25, y - 50};
}
```

optionBoxes

Purpose: stores and renders the Play and Exit menu buttons. It uses SDL_ttf to convert button text into textures. The destructor frees textTexture and exitTexture, which is important because SDL textures use memory outside ordinary C++ objects.

4. Runtime Flow

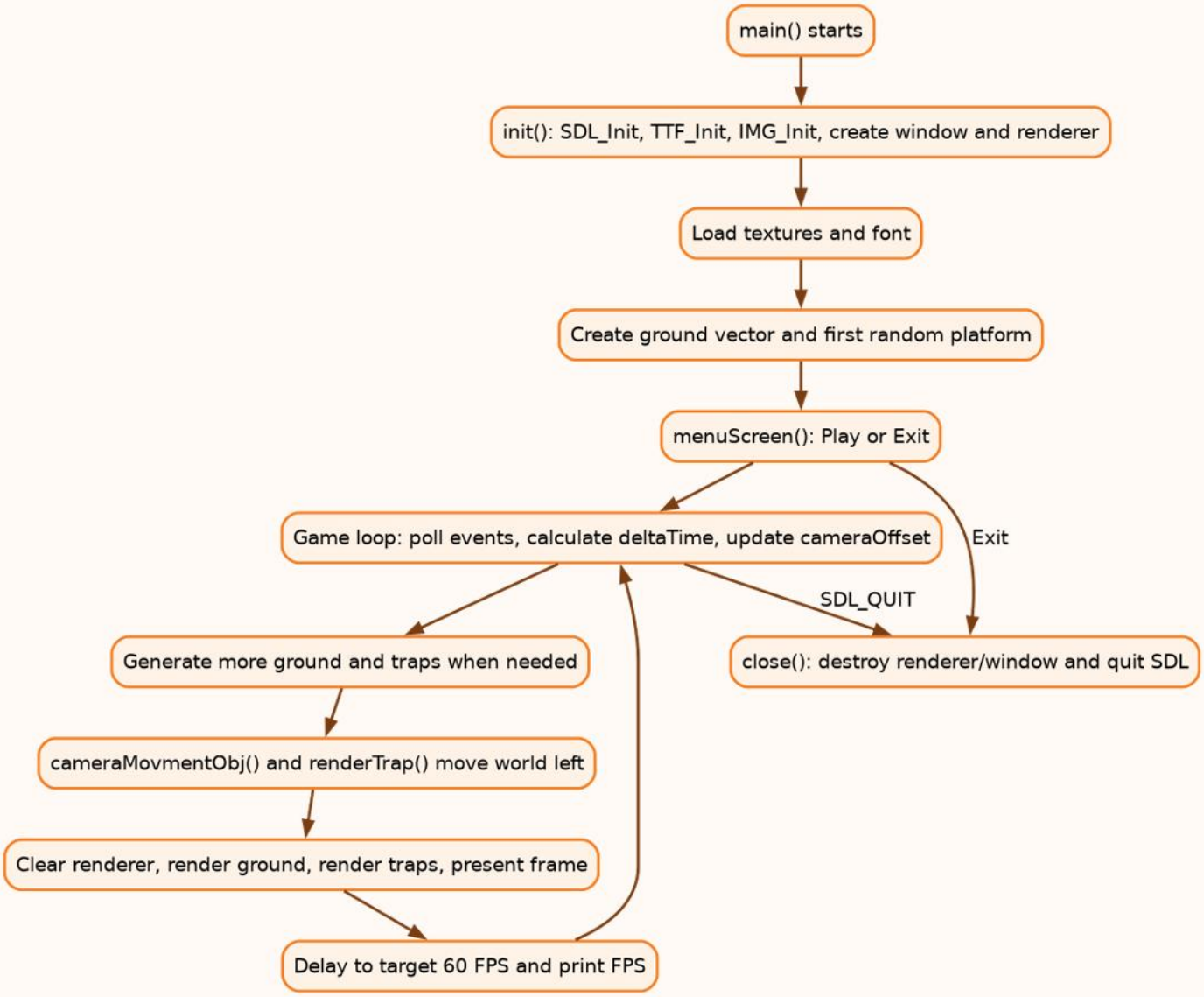

Figure 3: Program flow from initialization to game loop and cleanup

The game starts by creating SDL systems, then loading textures and fonts. The menu runs before the world update. After Play is clicked, each frame calculates deltaTime, updates the camera offset, creates new platforms if needed, deletes old objects, renders the frame, and delays to keep the frame rate near 60 FPS.

5. Core Algorithms

An algorithm is a clear step-by-step recipe. The Jumpa Verse code is easier to understand if each major part is read as a recipe instead of as one large program.

5.1 Main Program Algorithm

```
1. Create empty SDL_Window* and SDL_Renderer* pointers.
2. Call init(window, renderer).
3. If initialization fails, print an error and stop the program.
4. Load the ground texture, underground texture, and menu font.
5. Create the first ground platform with ground_generator().
6. Create the next random platform and push it into the ground vector.
7. Start the main loop while quit is false.
```

```
8. Measure frame time and calculate deltaTime.
9. Show menuScreen() until Play or Exit is selected.
10. Read SDL events such as window close and mouse/keyboard input.
11. Clear the renderer using the sky color.
12. Generate more ground if the vector has fewer than 30 platforms.
13. Remove old ground and traps from memory.
14. Move ground and trap x positions to create the camera illusion.
15. Render ground first, then traps, then present the completed frame.
16. Delay the frame if it finished faster than 1/60 second.
17. When the loop ends, destroy SDL resources and quit.
```

5.2 Platform Generation Algorithm

The first platform is created near the left side of the world. Later platforms are generated after the previous platform, with a small random gap. This keeps the level continuous but unpredictable.

```
1. Read the last platform from prevGround.back().
2. Choose a new x position after last.x + last.w.
3. Choose a y position near the previous y value.
4. Reject the y value if it is too high or too low.
5. Choose a random width and height.
6. Before returning the new platform, try to add traps to existing platforms.
7. Construct ground_Class(x, y, w, h, groundTex, ugTex).
8. Return the platform so main.cpp can push it into the vector.
```

5.3 Trap Placement Algorithm

A trap is not placed every time. The code uses a random chance so the stage feels less repetitive. Each platform also has trapCount so one platform does not become overloaded.

```
1. Pick a random x position inside the selected platform.
2. Pick a random chance from 0 to 100.
3. If the platform already has 4 traps, do not add another one.
4. If the chance is less than 5, create a trapSpike at that x value.
5. Store the triangle points p1, p2, and p3 inside the trapSpike object.
6. During rendering, draw the filled triangle and then its outline.
```

5.4 Frame Rendering Algorithm

SDL rendering is usually done in the same order every frame: clear, draw everything, then present. If you draw before clearing, the old frame may still be visible. If you forget present, the frame is drawn internally but never shown to the player.

```
1. SDL_SetRenderDrawColor(renderer, skyR, skyG, skyB, 255)
2. SDL_RenderClear(renderer)
3. for each platform: platform.render(renderer)
4. for each trap: trap.render(renderer)
5. SDL_RenderPresent(renderer)
```

6. SDL2 Basics

SDL2 is a cross-platform C library used from C++ to create windows, read input, draw graphics, play audio, and manage timing. It is low-level compared with a full game engine: SDL gives you the building blocks, while your code decides the game structure.

Think of SDL2 as the layer between your C++ code and the operating system. Without SDL2 you would need separate window, keyboard, mouse, and drawing code for Windows, Linux, and macOS. SDL2 gives one common API for those jobs.

6.1 Minimal SDL Program

```
#include <SDL2/SDL.h>

int main() {
SDL_Init(SDL_INIT_VIDEO);
SDL_Window* window = SDL_CreateWindow(
"Example", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
800, 600, SDL_WINDOW_SHOWN
);
SDL_Renderer* renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);

SDL_SetRenderDrawColor(renderer, 255, 128, 0, 255);
SDL_RenderClear(renderer);
SDL_RenderPresent(renderer);
SDL_Delay(2000);

SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Important idea: the renderer draws into a hidden back buffer. SDL_RenderPresent swaps that finished frame onto the screen.

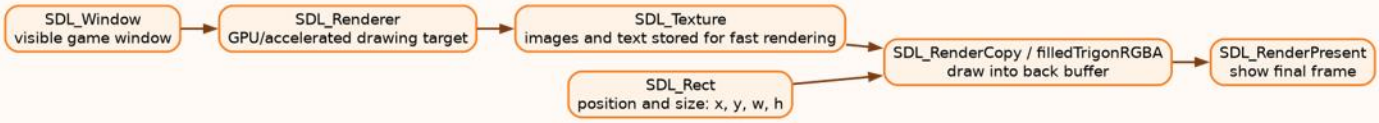

Figure 4: SDL rendering pipeline used by the project

6.1.1 Most Basic Working

The smallest SDL2 drawing program follows one rule: initialize SDL, create a window, create a renderer, draw into the renderer, present the renderer, wait briefly, then destroy everything. If one of those steps is skipped, the program may compile but show a black window, close instantly, or crash.

- Initialize first: SDL functions need SDL_Init before window or input work can happen.
- Create the window second: the renderer needs a valid window to draw into.
- Create the renderer third: all drawing commands target the renderer.
- Clear before drawing: this removes the previous frame.
- Present after drawing: this makes the completed frame visible.
- Destroy in reverse order: renderer first, then window, then SDL_Quit.

6.1.2 Common Errors In A Minimal SDL Program

- Window is null: SDL_CreateWindow failed. Print SDL_GetError() and check that SDL_Init(SDL_INIT_VIDEO) succeeded.
- Renderer is null: SDL_CreateRenderer failed. Try removing SDL_RENDERER_ACCELERATED or check graphics driver support.
- Black screen: you may be drawing but forgot SDL_RenderPresent, or you clear after drawing instead of before drawing.
- Program closes instantly: add an event loop or SDL_Delay so the window remains open long enough to see.
- Compile error for SDL.h: install SDL2 development headers and use the correct include path.
- Linker errors such as undefined reference: add -lSDL2 to the compile command after the source files.

6.2 SDL Coordinates and Rectangles

SDL uses a screen coordinate system where x increases to the right and y increases downward. The top-left corner is usually (0, 0). An SDL_Rect stores x, y, width, and height.

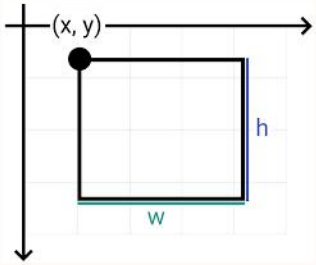

Figure 5: SDL2 coordinate reference from the project tutorial images

```
SDL_Rect box;
box.x = 100;  // left position
box.y = 200;  // top position
box.w = 80;   // width
box.h = 50;   // height

SDL_SetRenderDrawColor(renderer, 255, 128, 0, 255);
SDL_RenderFillRect(renderer, &box);
```

6.2.1 Common Coordinate Errors

- Object appears too low: remember that larger y means lower on the screen, not higher.
- Object appears off-screen: x or y may be negative or larger than the window width/height.
- Rectangle is invisible: w or h may be zero, negative, or too small to notice.
- Collision feels wrong: SDL_Rect uses the top-left corner plus width and height, so center-based math must be converted.

- Tile gaps appear: when building repeated SDL_Rect tiles, use consistent tile width and avoid rounding drift.

6.3 Events and Input

SDL_Event stores input and window messages. SDL_PollEvent reads all waiting events. This project uses SDL_QUIT to close the game and SDL_MOUSEBUTTONDOWN to detect menu clicks.

```
SDL_Event event;
while (SDL_PollEvent(&event)) {
if (event.type == SDL_QUIT) {
quit = true;
}
if (event.type == SDL_MOUSEBUTTONDOWN) {
int mx = event.button.x;
int my = event.button.y;
// compare mx/my with a button rectangle
}
}
```

6.3.1 Common Event Errors

- Window stops responding: SDL_PollEvent is not being called regularly inside the loop.
- Only one input works: use while(SDL_PollEvent(...)) instead of if(SDL_PollEvent(...)) so all waiting events are processed.
- Mouse clicks are ignored: compare event.button.x and event.button.y with the correct rectangle x/y/w/h values.
- Keyboard input repeats too fast: SDL_KEYDOWN can repeat when a key is held; check event.key.repeat if needed.
- Quit button does not close the game: set the loop's quit flag when event.type equals SDL_QUIT.

6.4 Timing and Delta Time

deltaTime is the number of seconds since the previous frame. Movement becomes smoother because objects move according to elapsed time instead of assuming every computer runs the same number of frames per second.

```
Uint32 currentTime = SDL_GetTicks();
float deltaTime = (currentTime - lastTime) / 1000.0f;
lastTime = currentTime;

float speed = 2.0f;
cameraOffset -= speed * deltaTime;
```

6.4.1 Common Timing Errors

- Movement depends on computer speed: movement is probably using a fixed amount per frame instead of speed * deltaTime.
- Huge jump after lag: clamp deltaTime, as this project does with if(deltaTime > 0.05f) deltaTime = 0.05f.

- FPS limit is unstable: calculate frame time after rendering and delay only for the remaining time.
- Object barely moves: the speed value is in units per second, so a very small speed may look frozen.
- Integer rounding removes movement: casting tiny float offsets to int can turn small movement into zero for a frame.

6.5 SDL Commands Used In Jumpa Verse

This section explains the basic SDL commands that appear in the project and the exact job of each command.

- SDL_Init(SDL_INIT_VIDEO): starts the SDL video system so the program can create a window and renderer.
- SDL_GetError(): returns a readable explanation when an SDL function fails.
- SDL_CreateWindow(...): asks the operating system to create the visible game window.
- SDL_CreateRenderer(...): creates the drawing context used for accelerated rendering.
- SDL_PollEvent(&event): reads window, mouse, and keyboard messages waiting in SDL's event queue.
- SDL_GetTicks(): returns milliseconds since SDL started; the project uses it for deltaTime and FPS.
- SDL_SetRenderDrawColor(...): selects the color used by rectangle, line, and clear operations.
- SDL_RenderClear(renderer): fills the back buffer with the current draw color.
- SDL_RenderCopy(renderer, texture, nullptr, &rect): copies a texture into a destination rectangle.
- SDL_RenderFillRect(renderer, &rect): draws a filled rectangle, used for menu blocks and the preview cube.
- SDL_RenderDrawRect(renderer, &rect): draws only the border of a rectangle.
- SDL_RenderDrawLine(renderer, x1, y1, x2, y2): draws the ground line in the menu animation.
- SDL_RenderPresent(renderer): displays the completed back buffer on screen.
- SDL_Delay(ms): pauses the program briefly; the project uses it to cap the frame rate.
- SDL_DestroyRenderer(renderer), SDL_DestroyWindow(window), SDL_Quit(): release SDL resources at the end.

6.5.1 Possible Error For Each Basic Command

- SDL_Init: returns a negative value on failure. Always check it before creating the window.
- SDL_CreateWindow: returns nullptr when the window cannot be created. Common reasons are invalid flags or video subsystem failure.

- SDL_CreateRenderer: returns nullptr if accelerated rendering is unavailable. A software renderer can be used as a fallback.
- SDL_SetRenderDrawColor: may fail if the renderer is invalid. If nothing draws, check that renderer is not nullptr.
- SDL_RenderClear: clears with the current draw color. If the whole screen is the wrong color, the previous draw color is probably still active.
- SDL_RenderCopy: returns an error if the texture or destination rectangle is invalid. A null texture is the most common cause.
- SDL_RenderFillRect: draws nothing useful when the rectangle has zero width or height.
- SDL_RenderPresent: must happen after drawing. Calling it before drawing shows the old frame.
- SDL_PollEvent: returns 0 when no events are waiting. That is normal and not an error.
- SDL_Delay: freezes the current thread. Do not use a long delay inside the active gameplay loop except for frame limiting.
- SDL_DestroyTexture/Renderer/Window: passing already-destroyed pointers can crash. Destroy once and set pointers to nullptr if reused.

6.6 Command Example: Drawing An Orange Rectangle

```
SDL_Rect playerPreview = {260, 955, 45, 45};

SDL_SetRenderDrawColor(renderer, 255, 127, 0, 255);
SDL_RenderFillRect(renderer, &playerPreview);

SDL_SetRenderDrawColor(renderer, 40, 22, 18, 255);
SDL_RenderDrawRect(renderer, &playerPreview);
```

The first color is orange, so SDL_RenderFillRect paints the inside of the rectangle orange. The second color is dark brown, so SDL_RenderDrawRect adds a border. This is the same idea used in menuScreen.cpp for the small animated cube.

6.7 The Most Basic SDL2 Frame

Every SDL2 frame in a simple 2D game can be understood as a sandwich: input at the start, update in the middle, draw at the end. Jumpa Verse follows this same pattern even though the code also handles menus, random platforms, traps, and FPS limiting.

```
while (!quit) {
// 1. INPUT
while (SDL_PollEvent(&event)) {
if (event.type == SDL_QUIT) quit = true;
}

// 2. UPDATE
cameraOffset -= speed * deltaTime;
cameraMovmentObj(ground, cameraOffset, trap);

// 3. DRAW
```

```
SDL_SetRenderDrawColor(renderer, 135, 206, 235, 255);
SDL_RenderClear(renderer);
// render objects here
SDL_RenderPresent(renderer);
}
```

7. SDL_image

SDL_image loads image files that plain SDL cannot load easily. In this project it is used for JPEG platform textures.

```
if (!(IMG_Init(IMG_INIT_JPG) & IMG_INIT_JPG)) {
std::cerr << IMG_GetError() << std::endl;
}

SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
SDL_Texture* ugTex = IMG_LoadTexture(renderer,
"Images_textures/underGround.jpeg");

SDL_RenderCopy(renderer, groundTex, nullptr, &destinationRect);
```

Best practice: load each texture once, reuse it many times, and destroy it when the program ends. The current ground_Class stores texture pointers as shared references.

7.1 SDL_image Possible Errors

- IMG_Init fails: the requested image format support is missing. Print IMG_GetError() for details.
- IMG_LoadTexture returns nullptr: the path is wrong, the file is missing, or the renderer is invalid.
- Texture shows as a blank area: check that the destination SDL_Rect is inside the window and has width/height.
- JPEG does not load: initialize with IMG_INIT_JPG and make sure SDL2_image was linked using -lSDL2_image.
- Memory usage grows: load textures once and reuse them instead of loading a new texture every frame.
- Crash during render: SDL_RenderCopy was given a null SDL_Texture pointer.

8. SDL_ttf

SDL_ttf loads TrueType fonts and creates text surfaces. The surface is then converted into an SDL_Texture for fast rendering.

```
TTF_Init();
TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 100);

SDL_Color orange = {255, 127, 0, 255};
SDL_Surface* surface = TTF_RenderText_Blended(font, "Play", orange);
SDL_Texture* textTexture = SDL_CreateTextureFromSurface(renderer, surface);
SDL_FreeSurface(surface);

SDL_RenderCopy(renderer, textTexture, nullptr, &textRect);
```

```
SDL_DestroyTexture(textTexture);
TTF_CloseFont(font);
TTF_Quit();
```

In menuScreen.cpp, optionBoxes uses this same technique for the Play and Exit labels. Hovering changes state, then the text texture is recreated with a different color.

8.1 SDL_ttf Possible Errors

- TTF_Init fails: SDL_ttf is not installed correctly or was not linked with -lSDL2_ttf.
- TTF_OpenFont returns nullptr: the font path is wrong, the font file is missing, or the font format is unsupported.
- TTF_RenderText_Blended returns nullptr: the font is null or the text render operation failed. Print TTF_GetError().
- Text is invisible: text color may match the background, or the destination rectangle may be off-screen.
- Text is blurry or too large: choose a better font size when calling TTF_OpenFont.
- Memory leak: every SDL_Surface from TTF_RenderText_Blended must be freed, and every created text texture must be destroyed.

9. SDL2_gfx

SDL2_gfx adds drawing primitives that SDL2 does not provide directly, such as filled triangles, circles, polygons, and anti-aliased lines. The project uses it for spike traps.

```
SDL2_GFXPRIMITIVES_SCOPE int trigonRGBA ( SDL_Renderer *  renderer,
                                          Sint16          x1,
                                          Sint16          y1,
                                          Sint16          x2,
                                          Sint16          y2,
                                          Sint16          x3,
                                          Sint16          y3,
                                          Uint8           r,
                                          Uint8           g,
                                          Uint8           b,
                                          Uint8           a
                                        )
```

Figure 6: Triangle color and point reference from the project tutorial images

```
#include <SDL2/SDL2_gfxPrimitives.h>

filledTrigonRGBA(renderer,
p1.x, p1.y,
p2.x, p2.y,
```

```
p3.x, p3.y,
92, 52, 40, 255);

trigonRGBA(renderer,
p1.x, p1.y,
p2.x, p2.y,
p3.x, p3.y,
40, 22, 18, 255);
```

The first call fills the triangle. The second call draws the outline. RGBA means red, green, blue, and alpha transparency.

9.1 SDL2_gfx Possible Errors

- Compile error for SDL2_gfxPrimitives.h: install SDL2_gfx development headers.
- Linker error for filledTrigonRGBA or trigonRGBA: add -lSDL2_gfx to the compile command.
- Triangle appears in the wrong place: check the three SDL_Point coordinates and remember y decreases upward.
- Triangle is invisible: alpha may be 0, color may match the background, or points may be outside the screen.
- Triangle outline does not align: use the same point coordinates for filledTrigonRGBA and trigonRGBA.

10. Game Camera and World Movement

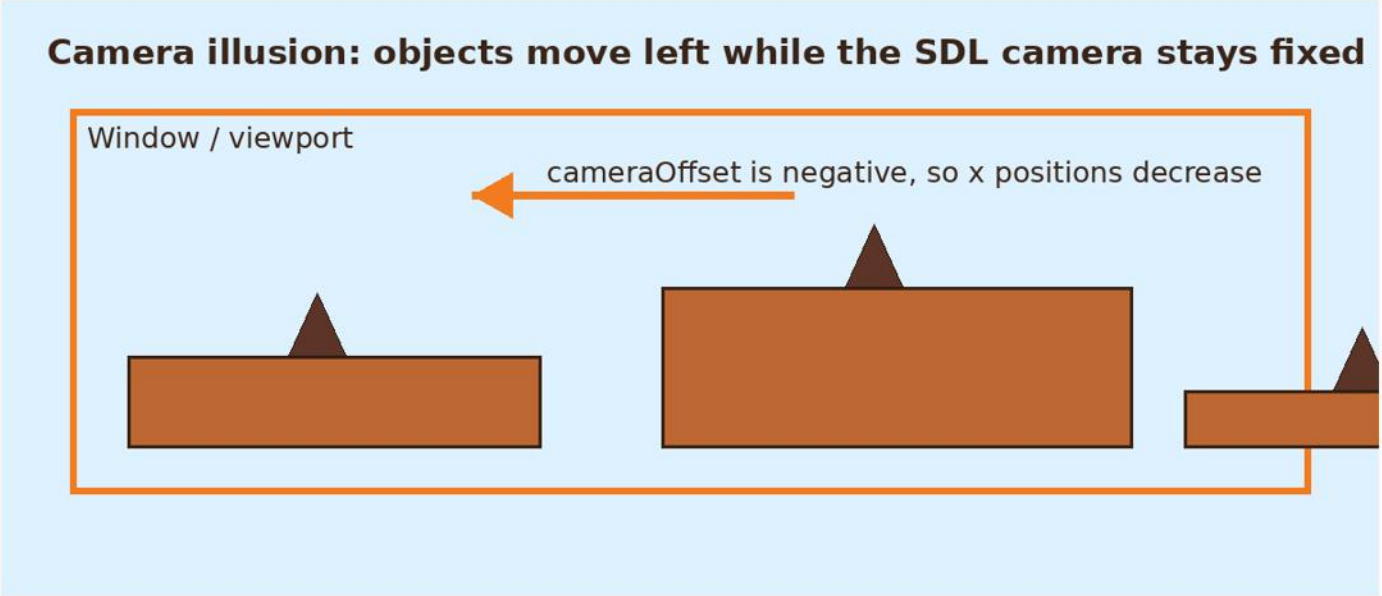

Figure 7: The camera illusion used by Jumpa Verse

The project keeps the SDL window fixed. Instead of moving an actual camera, the code moves ground and trap objects by changing their x positions. When x gets smaller, the object appears to travel left.

```
void cameraMovmentObj(std::vector<ground_Class>& ground, float cameraOffset,
std::vector<trapSpike>& trap) {
for (int i = 0; i < ground.size(); i++) {
ground[i].seterForElements(
```

```
ground[i].x + (int)cameraOffset,
ground[i].y,
ground[i].w,
ground[i].h
);
}
}
```

10.1 Camera Movement Errors

- World moves too fast: cameraOffset or speed is too large for the current deltaTime.
- World does not move: cameraOffset may round to 0 after casting to int.
- Objects jump instead of sliding: deltaTime may spike; clamp it before using it.
- Traps separate from platforms: ground and traps must be moved using compatible offset logic.
- Old objects remain forever: freeMemory and trapFreeMemory must remove off-screen objects.

11. Procedural Ground and Traps

ground_generator creates the first platform. groundRandomgenerator creates later platforms using random x, y, width, and height values based on the last platform. This makes the level continue without manually placing every platform.

```
std::uniform_int_distribution<> distx(
prevGround.back().x + prevGround.back().w + 120,
prevGround.back().x + prevGround.back().w + 200
);

std::uniform_int_distribution<> disty(
prevGround.back().y - 100,
prevGround.back().y + 600
);
```

trapTriangle may place a trap on a platform if a random chance succeeds and that platform has fewer than four traps. trapFreeMemory limits the trap vector so old traps do not grow forever.

11.1 Random Generation Errors

- Platform appears outside the screen: random y range needs limits, which the project checks with y < 800 and y > 100.
- Too many traps: lower the chance value or reduce the trapCount limit.
- No traps appear: the random chance may be too low or the loop may break before placing one.
- Platforms overlap badly: adjust the x gap range and the y range based on the previous platform.
- Game slows over time: erase old platforms and traps once they move beyond the useful area.

12. Memory and Cleanup

- SDL_DestroyRenderer(renderer) frees the renderer.
- SDL_DestroyWindow(window) frees the window.
- SDL_DestroyTexture(texture) should be called for textures when no longer needed.
- SDL_FreeSurface(surface) is required after converting a text surface into a texture.
- IMG_Quit(), TTF_Quit(), and SDL_Quit() shut down SDL libraries.
- Vectors such as ground and trap should erase objects that are off-screen to avoid unnecessary memory use.

```
void close(SDL_Window* window, SDL_Renderer* renderer) {
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
IMG_Quit();
SDL_Quit();
}
```

Recommended improvement for this project: close() should also call TTF_Quit(), and main.cpp should destroy groundTex and ugTex after the game loop.

12.1 Cleanup Errors

- Memory leak: surfaces, textures, fonts, renderer, and window are not destroyed.
- Double free crash: the same SDL resource is destroyed twice.
- Use-after-free crash: a texture is destroyed while an object still tries to render it.
- TTF resources remain active: call TTF_CloseFont for opened fonts and TTF_Quit before final SDL_Quit.
- Shutdown order confusion: destroy objects that depend on renderer before destroying the renderer.

13. Beginner-Friendly Build Notes

A typical Linux compile command for this project should link SDL2, SDL2_image, SDL2_ttf, and SDL2_gfx.

```
g++ main.cpp Error_Handlers.cpp ground.cpp camera.cpp traps.cpp menuScreen.cpp \
-o test \
-lSDL2 -lSDL2_image -lSDL2_ttf -lSDL2_gfx
```

If headers are missing, install the development packages for SDL2, SDL2_image, SDL2_ttf, and SDL2_gfx for your Linux distribution.

13.1 Build Errors And Fixes

- fatal error: SDL2/SDL.h: No such file or directory: SDL2 headers are not installed or include paths are wrong.
- undefined reference to SDL_Init: the SDL2 library was not linked; add -lSDL2 after source files.
- undefined reference to IMG_LoadTexture: add -lSDL2_image.
- undefined reference to TTF_OpenFont: add -lSDL2_ttf.

- undefined reference to filledTrigonRGBA: add -lSDL2_gfx.
- Runtime file not found: run the game from the project root so Images_textures and debrosee-font paths resolve.

14. How To Read The Code

- Start from main.cpp because it shows the complete lifecycle.
- Read init() next to understand SDL setup.
- Read object.h to understand what data each object stores.
- Read ground.cpp and traps.cpp to understand level generation.
- Read camera.cpp to understand why objects move left.
- Read menuScreen.cpp to understand SDL mouse input and SDL_ttf text rendering.

15. Suggested Improvements

- Add a player class with position, velocity, jump physics, collision checks, and animation state.
- Destroy loaded textures and close the font before exiting.
- Move constants such as window size, tile size, and colors into a config header.
- Use size_t for vector indexes where possible.
- Separate rendering, updating, and input into clearer functions as the game grows.
- Add collision detection between player hitBox and trapSpike/ground_Class.

16. Quick Glossary

- Window: the operating-system window where the game appears.
- Renderer: SDL object used to draw images, rectangles, and primitives.
- Texture: GPU-friendly image used for fast rendering.
- Surface: CPU-side image data, often temporary before creating a texture.
- Rect: x/y/w/h rectangle used for position and size.
- Event: keyboard, mouse, quit, and other input message.
- Delta time: elapsed time between frames.
- RGBA: red, green, blue, alpha color values.